

Hormigas Artificiales (ACO)

Ricardo Contreras A.

María Angélica Pinninghoff J.

Magister en Inteligencia Artificial

Universidad Adolfo Ibáñez

Noviembre 2023

1.- Introducción

La idea de las hormigas artificiales y sus primeros algoritmos fueron desarrollados por Marco Dorigo y Gianni di Caro en el año 1991 en la Université Libre de Bruxelles. Estos investigadores buscaban una heurística para resolver problemas de optimización representados por el problema del vendedor viajero (TSP: Travel Salesman Problem) y posteriormente, para problemas tipo ruteamiento vehicular (VRP: Vehicular Routing Problem). El problema del vendedor viajero consiste en un vendedor que debe visitar un conjunto de ciudades o clientes sin repetir, en el menor tiempo posible. El problema del ruteamiento vehicular es similar al TSP, pero en este caso se trata de transportar mercadería en camiones o algún tipo de transporte. En este último caso, el medio de transporte tiene una capacidad definida y los clientes tienen requerimientos de mercadería, luego no sólo hay que minimizar el recorrido, sino también el número de vehículos de transporte.

Después de varios intentos de algoritmos, el que fue publicado como definitivo por los autores fue el algoritmo AS (Ant Systems). Este algoritmo presenta resultados comparables a otras técnicas de optimización en muchos problemas a los cuales ha sido aplicado.

Con el tiempo se han desarrollado distintos algoritmos basados en hormigas artificiales que son variaciones del algoritmo AS. Este conjunto de algoritmos es llamado ACO (Ant Colony Optimization).

Los algoritmos ACO están inspirados en la forma en que las hormigas naturales encuentran el camino mínimo entre su hormiguero y una fuente de comida. ¿Cómo lo hacen? Lo hacen mediante una comunicación indirecta. En este

tipo de comunicación indirecta, las hormigas expelen una sustancia llamada feromona, luego al pasar por algún lugar dejan un rastro de feromona, el cual, con el tiempo se va evaporando, pero si otra hormiga pasa por el mismo lugar el olor se refuerza. De esa forma las hormigas naturales se comunican en forma indirecta a través del olor. Entonces, en la búsqueda de comida las hormigas prefieren seguir el rastro con olor más fuerte. Así, en un principio hacen una exploración al azar y en paralelo del ambiente, para luego ir acotando los caminos recorridos y terminar en el camino más corto, que es la hilera de hormigas que vemos habitualmente.

Lo que las hormigas artificiales hacen es una imitación de la situación descrita anteriormente, para ello es necesario definir un ambiente, una colonia de hormigas, cada hormiga en particular con sus características y luego el algoritmo que las hará actuar. Más adelante se verá su formalización.

Una hormiga por sí sola no resuelve ningún problema, el problema es resuelto por la colonia de hormigas, debido a esto es que los algoritmos ACO, en Inteligencia Artificial, son considerados dentro de la **inteligencia de enjambre** o swarm intelligence.

2.- Descripción de las hormigas artificiales

Se puede resumir los algoritmos ACO como un conjunto de agentes concurrentes y asincrónicos. Concurrentes porque todas las hormigas de la colonia recorren el ambiente al mismo tiempo y asincrónicos porque no están coordinadas ni controladas por una entidad superior, son autónomas, cada hormiga recorre el ambiente por su cuenta, lo único que la guía es la información, sobre el ambiente, que encuentra en su camino. Cada hormiga en su caminar va construyendo una solución al problema que se está tratando de resolver.

Para moverse en su recorrido del ambiente, una hormiga considera dos tipos de información: una local que consiste en la cantidad de feromona que hay en los puntos a los cuales se puede mover en su recorrido y una global que es un valor heurístico que depende del problema particular. Considerando estos dos valores como una probabilidad, la hormiga decide a qué punto moverse dentro del ambiente. Además, cada hormiga debe tener una memoria donde va guardando el camino recorrido, con el fin de no repetirlo.

Una hormiga es un agente reactivo y autónomo que obedece a reglas simples de movimiento. Los caminos recorridos y por lo tanto la construcción de la solución son emergentes, esto es, no son predeterminados. El conjunto de hormigas (la colonia) actuando colaborativamente, a través de la feromona y la heurística, consiguen construir buenas soluciones al problema.

Para aplicar el enfoque ACO a algún problema, es necesario tener algunas consideraciones.

El **ambiente** que van a recorrer las hormigas (i.e. el espacio solución del problema) debe ser representado por un grafo $G=(V,E)$ [cita grafos]. Un grafo G es una red compuesta por vértices V , también llamados nodos, y por aristas E que comunican los vértices. Esto es así para imitar la búsqueda de un camino.

La **feromona** y la **información heurística** son asociadas a las aristas del grafo, por lo tanto es necesario definir estructuras de datos para guardar y actualizar esta información. Normalmente se usa matrices para representar tanto el ambiente (o espacio solución), como para la información de la feromona y de la heurística.

El algoritmo de hormigas artificiales es un proceso iterativo. En cada iteración cada hormiga de la colonia construye una solución al problema y entrega la mejor solución encontrada y luego actualiza los rastros de feromona. La actualización de feromona consiste en agregar una cantidad de feromona a las aristas por donde pasó una hormiga y disminuir una cierta cantidad fija a todas las aristas del grafo, simulando el efecto de evaporación de la feromona en el ambiente. La cantidad de feromona agregada depende de la calidad de la solución encontrada por esa hormiga.

En el proceso de construcción de una solución, inicialmente cada hormiga de la colonia es ubicada al azar en los vértices del grafo. Cada hormiga tiene posibilidad de **moverse** desde un vértice v del grafo, donde está ubicada, hacia otros vértices que están conectados con v por una arista e y no están en el camino ya recorrido de la hormiga. El movimiento es probabilístico, la hormiga se mueve a los vértices adyacentes con una **probabilidad** que considera la **feromona** y la **heurística** de las aristas que los unen, luego la hormiga decide qué arista cruzar en su camino usando las probabilidades de todas las aristas que puede cruzar. El movimiento de cada hormiga de la colonia se realiza de la misma forma. Luego **este proceso se repite hasta que cada hormiga haya construido una solución**.

Una vez terminado el proceso de construcción de la solución, cada solución es evaluada de acuerdo a una función de evaluación que es propia del problema. Con estos valores se calcula el aumento de feromona en las aristas recorridas. Además se guarda la mejor solución encontrada por la colonia.

3.- Algoritmo ACO Básico

Estructuras de datos básicas:

- Una **matriz G** para representar el grafo del ambiente. El tamaño de esta matriz depende del espacio solución del problema. Por ejemplo, el número de ciudades o clientes en el ejemplo dado. Esta matriz es una matriz cuadrada,
- Una matriz de **feromona τ** . Esta matriz tiene el mismo tamaño de la matriz G; es actualizada en cada iteración, por lo tanto su contenido son valores variables. Los valores no dependen del problema, son propios del enfoque ACO. Dependen del valor inicial de feromona, de la tasa de evaporación y del total de feromona repartido entre las hormigas en forma proporcional a la calidad de la solución encontrada.
- Una matriz de **heurística η** . Esta matriz también tiene el tamaño de G, sus valores son fijos. Contiene información propia del problema, por ejemplo distancia entre ciudades o clientes.
- Hormiga **k**. Esta es una estructura que debe contener una **memoria** para ir guardando las aristas del camino recorrido por la hormiga. Además debe contener funciones que le permitan moverse: cálculo de **probabilidad** de las aristas conectadas con el vértice actual y no recorridas por la hormiga **k**, y capacidad de **elegir el siguiente vértice** usando estocásticamente las probabilidades de las aristas posibles de cruzar. La fórmula del cálculo de probabilidades para cada arista posible de cruzar es la siguiente:

$$P_{ij}(k) = \begin{cases} \frac{\tau_{ij}^{\alpha} \cdot \eta_{ij}^{\beta}}{\sum_{h \in NoVisitadas} \tau_{ih}^{\alpha} \cdot \eta_{ih}^{\beta}} & j \in NoVisitadas \\ 0 & en \text{ otro caso} \end{cases}$$

Donde **k** es la k-ésima hormiga de la colonia; **P_{ij}** es la probabilidad de la arista (i,j) posible de visitar por la hormiga k; **τ_{ij}** es la cantidad de feromona que tiene la arista (i,j); **η_{ij}** es el valor heurístico de la arista (i,j); α y β son valores que permiten darle una ponderación mayor, ya sea a la heurística o a la feromona, si ambas son iguales a 1, tienen la misma ponderación.

Funciones generales:

- Una **función de evaluación (fe)**. Esta función es propia del problema y su fin es entregar un valor que indique la calidad de la solución encontrada por cada hormiga de la colonia, en cada iteración.
- **Actualización de la feromona** en cada arista. Esta incluye el aumento y la evaporación. En el aumento de feromona se tiene:

$$\Delta\tau_{ij}(t+1) = \sum_{k=1}^{NroAnts} \Delta^k\tau_{ij}$$

Donde el **aumento de feromona** de la arista (i,j) está dado por la suma de todas las hormigas (k) de la colonia que pasaron por la arista (i,j). El aumento de feromona de cada hormiga, depende de la calidad de su solución, por lo tanto hay una cantidad **Q** de feromona total que se reparte entre las hormigas de tal forma que la mejor solución reparta la mayor cantidad en cada arista de su solución. La fórmula para repartir depende del problema de que se trate, por ejemplo en el vendedor viajero, como se trata de minimizar las distancias, se calcula **Q/ fe(k)**, pero no en todos los problemas es igual.

La **evaporación de feromona** es una tasa de disminución fija (ρ) de todas las aristas de la matriz de feromona sin importar la cantidad de hormigas que hayan pasado por ellas:

$$\tau_{ij} = (1 - \rho) * \tau_{ij}$$

Luego la **fórmula completa de actualización** de feromona queda:

$$\tau_{ij} = (1 - \rho) * \tau_{ij} + \Delta\tau_{ij}$$

Ejemplo

Supóngase que un vendedor viajero debe visitar 5 ciudades. Debe visitarlas todas sin repetirlas. Las ciudades son A, B, C, D y E. Las distancias entre cada par de ciudades está dada en el grafo representado por la matriz η , y los valores de feromona actuales están dados en la matriz τ . La función de evaluación (**fe**), que mide la calidad de la solución, en este problema está dada por la suma de las distancias recorridas, ya que la mejor solución es la que recorre la menor distancia en la visita a todas las ciudades.

η	A	B	C	D	E
A	0	60	20	30	40
B	60	0	40	20	50
C	20	10	0	40	10
D	30	20	40	0	30
E	40	50	10	30	0

τ	A	B	C	D	E
A	0	10	10	10	10
B	10	0	10	10	10
C	10	10	0	10	10
D	10	10	10	0	10
E	10	10	10	10	0

Inicialmente todas las aristas tienen la misma cantidad de feromona τ_0 . Sólo se verá el movimiento de la hormiga k y suponga que esta hormiga está comenzando su recorrido y fue colocada en el vértice C . Se supone también que la heurística (distancias en este caso) y la feromona tienen el mismo peso en la probabilidad de moverse, por lo tanto $\alpha=\beta=1$ en la fórmula de la probabilidad. Entonces la hormiga k se puede mover a los vértices A , B , D y E , luego se debe calcular la probabilidad de cada uno de estas aristas:

$$P_{(C,A)}(k) = (\eta_{(C,A)} * \tau_{(C,A)}) / (\eta_{(C,B)} * \tau_{(C,B)} + \eta_{(C,D)} * \tau_{(C,D)} + \eta_{(C,E)} * \tau_{(C,E)})$$

$$P_{(C,A)}(k) = (20*10) / (10*10 + 40*10 + 10*10);$$

$$\text{Luego, } P_{(C,B)}(k) = (\eta_{(C,B)} * \tau_{(C,B)}) / (\eta_{(C,A)} * \tau_{(C,A)} + \eta_{(C,D)} * \tau_{(C,D)} + \eta_{(C,E)} * \tau_{(C,E)})$$

De la misma forma se calcula la probabilidad para las aristas (C,D) y (C,E) .

Una vez calculadas las probabilidades, la hormiga k elige una arista al azar, donde este azar está influido por la probabilidad. Para la elección se puede usar una **ruleta** como en los Algoritmos Genéticos [cita Ruleta].

Si ya todas las hormigas dieron un paso y suponga que la hormiga k eligió el vértice E , entonces en su memoria tiene guardado $[C-E]$ y las posibilidades de moverse son las aristas (E,A) , (E,B) , (E,D) .

Ahora suponga que todas las hormigas generaron una solución y que la hormiga k hizo el recorrido $[C-E-D-B-A-C]$. Entonces $fe(k) = 10+30+20+60+20 = 140$. Y se calcula de la misma forma para la solución entregada por cada hormiga. Suponga también la cantidad de feromona que aumentan las hormigas $Q=1000$ y que la tasa de evaporación de feromona $p=0,1$. En este caso la hormiga k aumenta $1000/140=7$ unidades de feromona en cada arista de su solución. Para facilitar el ejemplo, supongamos que sólo hay 3 hormigas: k , l , m , y las soluciones entregadas por l y m son $l=[A-B-C-D-E-A]$ y $m=[E-C-B-D-A-E]$, entonces $fe(l)=(60+40+40+30+40)=210$ y $fe(m)=(10+10+20+30+40)=110$. La hormiga l aumenta $1000/210=5$ y m aumenta $1000/110=9$ unidades.

Como las aristas del grafo contienen distancias, la distancia de A a B es la misma que de B a A , por lo tanto basta con modificar las siguientes aristas (en la matriz es necesario modificarlas todas):

	(A,B)	(A,C)	(A,D)	(A,E)	(B,C)	(B,D)	(B,E)	(C,D)	(C,E)	(D,E)
k:	7	7	0	0	0	7	0	0	7	7
l:	5	0	0	5	5	0	0	5	0	5
m	0	0	9	9	9	9	0	0	9	0
Suma:	12	7	9	14	14	16	0	5	16	12

Como $\rho=0,1$; $(1-0,1)=0,9$ y como todas las aristas, en este momento, tienen 10, la evaporación queda en 9 para cada arista. Luego la actualización de feromona queda:

(A,B)	(A,C)	(A,D)	(A,E)	(B,C)	(B,D)	(B,E)	(C,D)	(C,E)	(D,E)
9+12	9+7	9+9	9+14	9+14	9+19	9+0	9+5	9+16	9+12
21	16	18	23	23	27	9	14	25	21

La matriz de feromona después de la actualización queda:

τ	A	B	C	D	E
A	0	21	16	18	23
B	21	0	23	27	9
C	16	23	0	14	25
D	18	27	10	0	21
E	23	9	10	10	0

Estos serán los valores de feromona usados por las hormigas para calcular la probabilidad de moverse en la siguiente colonia (siguiente iteración).

- **Parámetros:**

Los parámetros son valores fijos para cada ejecución del algoritmo, pero que deben ser calibrados antes de la puesta en marcha del algoritmo. Esta calibración sólo puede ser hecha probando las distintas combinaciones de valores de parámetros. Puede ser un proceso más o menos largo y en cierta forma depende del problema, por eso es un proceso previo a la puesta en marcha.

Los principales parámetros son:

- + Número de colonias, esto es igual al número de iteraciones.
- + Número de hormigas de la colonia

+ Valor de la feromona inicial. Este es el valor inicial para la matriz de feromona. Todos los vértices se inicializan con el mismo valor.

+ Tasa de evaporación ρ

+ Valor de la cantidad de feromona a repartir en el incremento en cada arista Q , según la calidad de la solución entregada por cada hormiga.

- Procedimiento

Las variables auxiliares usadas son:

+ c para contador del número de colonias

+ k para contador de número de hormigas

Algoritmo AS

```
{Inicializar();
```

```
for c=1 to Nro_colonias
```

```
{
```

```
for k=1 to Nro_hormigas
```

```
{
```

```
ant-k construye solución k
```

```
}
```

```
Guardar la mejor solución;
```

```
Actualizar Rastro Feromona  $//\tau_{ij}$ ;
```

```
{Evaporar;
```

```
Agregar}
```

```
Reubicar hormigas para el próximo ciclo;
```

```
}
```

```
Imprimir la mejor solución encontrada
```

```
}
```

Notas:

“**Inicializar**” implica darle valores a los parámetros y ubicar al azar las hormigas en el grafo.

“**ant-k construye solución k**” es un proceso iterativo donde la hormiga recorre el grafo formando su solución. Para dar cada paso usa la fórmula de la probabilidad.

“**Actualizar Rastro Feromona**” en el algoritmo **AS** usa las fórmulas anteriores para actualizar el rastro de feromona.

“**Reubicar hormigas para próximo ciclo**” implica borrar la solución encontrada en cada hormiga, luego ubicarlas al azar en el grafo para comenzar un nuevo ciclo, esta vez con los valores de feromona actualizados.

4.- Otros Algoritmos ACO

Los demás algoritmos ACO, son modificaciones al algoritmo AS, que surgieron posteriormente. Surgen como respuesta a ciertos problemas observados en AS y básicamente se diferencian en cómo usan y/o modifican el rastro de feromona.

MinMaxAS: Ant System con valores Mínimos y Máximos para los valores del rastro de feromona.

¿Qué puede ocurrir si no se controlan los valores del rastro de feromona?

Si la feromona de una arista llega a 0, como la fórmula de la probabilidad es una multiplicación, la probabilidad de elegir esa arista es igual a cero, por lo tanto nunca sería elegida. Por otro lado, si algunas aristas tienen un valor muy alto, la probabilidad de ser elegidas puede ser cercana a 1 y siempre serían elegidas, y no necesariamente puede ser el mejor camino. A esta situación se le llama óptimo local y no es deseable, lo que realmente interesa es obtener el óptimo global o una solución muy cercana a este último óptimo. Por ello se limita el máximo y el mínimo de feromona que puede tener una arista.

Estos valores máximo y mínimo son parámetros que se agregan a este algoritmo y deben ser probados como tales.

AS-rank: Ant System que usa un ranking de las mejores soluciones para realizar la actualización del rastro.

Aquí cambia un poco el aumento de feromona, la evaporación es igual. El aumento de feromona está dado por la siguiente fórmula:

$$\Delta \tau_{ij}(t+1) = \sum_{r=1}^{w-1} (w-r) \Delta^r \tau_{ij} + w \Delta^b \tau_{ij}$$

Donde w es un peso que es total de soluciones rankeadas; r es el índice del ranking, por ejemplo, si se rankean las 50 mejores soluciones, $w=50$ y r toma los valores entre 1 y $w-1$; y b es una arista que está en la mejor solución. También es un parámetro para este algoritmo, debe ser probado. Luego, para cada arista la fórmula de actualización queda:

$$\tau_{ij}(t+1) = \rho \cdot \tau_{ij}(t) + \Delta \tau_{ij}(t+1)$$

AS-elitism: Ant System que usa complementariamente la mejor solución encontrada hasta el momento para dar un resfuerzo adicional.

$$\Delta \tau_{ij}(t+1) = \sum_{k=1}^{NroAnts} \Delta^k \tau_{ij} + e \Delta^b \tau_{ij}$$

e: peso

b: mejor camino (best)

$$\tau_{ij}(t+1) = \rho \cdot \tau_{ij}(t) + \Delta \tau_{ij}(t+1)$$

Ant Colony System (ACS): Un algoritmo ACO que es una extensión de un AS e introduce: Un esquema local y global de actualización del rastro de feromona y una forma un poco diferente de elegir el próximo paso.

1. Actualización del rastro

- LOCAL:

Cada vez que una hormiga avanza en el grafo deja en la arista por la que pasó, un rastro muy pequeño de feromona (ξ), sin considerar la calidad de la solución. En este caso, la cantidad de feromona que se aumenta es una pequeña porción (ξ) de la feromona inicial (τ_0) con independencia de la cantidad de feromona que tenga la arista en ese momento. Por lo tanto, esta es una cantidad fija.

$$\tau_{ij} = (1 - \xi) \cdot \tau_{ij} + \xi \tau_0$$

- **GLOBAL:** Igual que en AS, es decir, después que termina un ciclo.

$$\tau_{ij}(t+1) = (1 - \rho) \cdot \tau_{ij}(t) + \rho \Delta \tau_{ij}^b(t+1)$$

2. Selección de la próxima componente de la solución:

La selección del próximo paso puede ser determinista (elige directamente la mejor arista) o probabilista (usando la fórmula de las probabilidades). Esto depende de un parámetro llamado q_0 . El procedimiento es el siguiente: se elige un número q al azar y si este valor es menor que q_0 , se elige de inmediato la mejor arista; si no lo es, se calculan las probabilidades y se elige igual que en AS, pero con $\alpha=1$.

Luego, la fórmula de selección es la siguiente:

$$j = \begin{cases} \arg \max_{l \in N_i} \{ \tau_{il} [\eta_{il}]^\beta \} & \text{si } q \leq q_0 \\ P_{ij} & \text{en otro caso} \end{cases}$$

Ahora bien, si se desea que la mayoría de selección de pasos sea determinista, q_0 debe ser un valor alto, de lo contrario la mayoría de las selecciones de pasos será probabilista. Esto también es materia de prueba.

Algoritmo ACS

```
{Inicializar();  
for c=1 to Nro_colonias  
{  
  for k=1 to Nro_hormigas  
  {  
    ant-k construye solución k; Actualización local de feromona  
  }  
  Guardar la mejor solución;  
  Realizar un ranking;  
  Actualizar Rastro Feromona //  $\tau_{ij}$ ;  
  { Evaporar;  
    Agregar }  
  Reubicar hormigas para el próximo ciclo;  
}
```

Imprimir la mejor solución encontrada

}

Bibliografía

Dorigo, M., Stützle, T. Ant Colony Optimization. MIT Press, 2004.